

Automated Docker Image Deployment to Amazon ECR with Jenkins and Lambda Integration

This project automates the process of building a Docker image, pushing it to Amazon ECR using a Jenkins CI/CD pipeline, and triggering an AWS Lambda function via EventBridge to send notifications through SNS and log deployment details in DynamoDB.

Technologies Used: Docker, Amazon ECR, Jenkins, AWS Lambda, Amazon SNS, Amazon DynamoDB, EventBridge, GitHub

Project Architecture

The pipeline works like this: a developer pushes code to GitHub, Jenkins detects the change via a webhook, builds a Docker image, and pushes it to Amazon ECR. When the image lands in ECR, EventBridge picks up the event and triggers a Lambda function that sends an email notification via SNS and writes a log entry to DynamoDB.

Step 1 — Create the Main DevOps EC2 Server

Go to AWS Console and create an EC2 instance with the following settings:

- **Name:** Main Devops-Server
- **OS:** Ubuntu
- **Instance Type:** t2.micro
- **Storage:** 15 GB

This server is where you will build and push your Docker images.

Step 2 — Create an IAM Role and Attach It to the EC2

Create an IAM role with the necessary permissions and attach it to the DevOps EC2 instance. This allows the server to interact with AWS services like ECR without needing static credentials.

Step 3 — Install Docker and AWS CLI on the DevOps Server

SSH into your EC2 instance and run the following commands.

Install Docker:

```
sudo apt update -y
sudo apt install docker.io -y
docker --version
```

```
sudo systemctl start docker
sudo systemctl enable docker
```

Install AWS CLI by following the official AWS documentation (download the zip installer).

Once done, verify your identity:

```
aws sts get-caller-identity
```

You should see your account ID and the assumed role name in the output. This confirms the IAM role is working correctly.

Step 4 — Create an ECR Repository

Go to the AWS Console and navigate to Amazon ECR. Create a new private repository. This is where your Docker images will be stored.

Step 5 — Create the Docker Project on the EC2

On the DevOps server, create a project folder and the necessary files.

```
mkdir sahyadri
cd sahyadri
```

Create app.py — your Flask application code (add from your repository).

Create requirements.txt:

```
flask
```

Create Dockerfile:

```
FROM python:3.9-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```

Build and test the Docker image:

```
docker build -t myapp:v1 .
docker images
docker run -d -p 5000:5000 myapp:v1
```

Open a browser and go to http://YOUR_EC2_PUBLIC_IP:5000 to confirm the app is running.

Step 6 — Log In to ECR from Docker

Authenticate your Docker client to ECR so you can push images:

```
aws ecr get-login-password --region ap-south-1 \
| docker login --username AWS \
--password-stdin 964177142854.dkr.ecr.ap-south-1.amazonaws.com
```

Step 7 — Tag and Push the Image to ECR

```
docker tag myapp:v1 964177142854.dkr.ecr.ap-south-1.amazonaws.com/docker-ci-cd-repo:v1
docker push 964177142854.dkr.ecr.ap-south-1.amazonaws.com/docker-ci-cd-repo:v1
```

After this, you should see the image listed inside your ECR repository in the AWS Console.

Step 8 — Create the Jenkins EC2 Server

Create a second EC2 instance dedicated to Jenkins:

- **Name:** Jenkins-Server
- **OS:** Ubuntu
- **Instance Type:** t3.small
- **Storage:** 20 GB

Install Java (required for Jenkins):

```
sudo apt update
sudo apt install fontconfig openjdk-21-jre
java -version
```

Install Docker (same steps as Step 3).

Install AWS CLI (same as Step 3).

Install Jenkins:

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key

echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
sudo apt install jenkins
```

Step 9 — Create an IAM Role for Jenkins and Attach It

Create a new IAM role called `jenkins_project_role` and attach the following policy:

- **AmazonEC2ContainerRegistryFullAccess**

Attach this role to the Jenkins EC2 instance so Jenkins can push images to ECR.

Step 10 — Configure Jenkins

Open Jenkins in your browser at `http://YOUR_JENKINS_IP:8080` and complete the initial setup.

Phase 1 — Push Your Application Code to GitHub

On the DevOps server, inside your project folder:

```
git init
git add .
git commit -m "push my internship project no 4"
git remote add origin YOUR_GITHUB_REPO_URL
git push origin master
```

Use your GitHub username and a personal access token when prompted.

Phase 2 — Install Required Jenkins Plugins

Go to **Manage Jenkins** → **Plugins** and install the following:

Plugin	Purpose
Git	GitHub integration
Pipeline	CI/CD pipeline
Docker Pipeline	Docker commands in pipeline

Plugin	Purpose
GitHub Integration	Webhook trigger
Blue Ocean	Better pipeline UI

Phase 3 — Create a Jenkins Pipeline

Create a new Pipeline job in Jenkins and write a Jenkinsfile that handles building the Docker image and pushing it to ECR. The pipeline should authenticate to ECR, build the image, tag it, and push it.

Phase 4 — Set Up a GitHub Webhook

This makes Jenkins automatically trigger a build whenever you push code.

Go to your GitHub repository and click **Settings** → **Webhooks** → **Add Webhook**.

- **Payload URL:** `http://YOUR_JENKINS_IP:8080/github-webhook/`
- **Content type:** `application/json`
- **Event:** Just the push event

Save the webhook. Now every time you push code to GitHub, the Jenkins pipeline will run automatically.

Step 11 — Create an SNS Topic for Notifications

Go to **AWS Console** → **SNS** → **Topics** and create a new Standard topic. Then create a subscription for your email address inside that topic.

AWS will send a confirmation email. Click the link in that email to confirm your subscription. You will not receive notifications until you confirm.

Step 12 — Create a DynamoDB Table for Logging

Go to **AWS Console** → **DynamoDB** → **Create Table** with the following settings:

Field	Value
Table Name	deployment-logs
Partition Key	id (String)

This table will store a log entry every time a deployment happens.

Step 13 — Create a Lambda Function

Go to **AWS Console** → **Lambda** → **Create Function** with the following settings:

Setting	Value
Name	ecr-image-trigger

Setting	Value
Runtime	Python 3.12
Architecture	x86_64

After creating the function, go to **Configuration** → **Permissions** → **Execution Role**, click on the role name, and attach these two policies:

- **AmazonSNSFullAccess**
- **AmazonDynamoDBFullAccess**

Step 14 — Add the Lambda Function Code

Replace the default Lambda code with the following:

```
import json
import boto3
import uuid
from datetime import datetime

sns = boto3.client('sns')
dynamodb = boto3.resource('dynamodb')

TOPIC_ARN = "PASTE_YOUR_SNS_TOPIC_ARN_HERE"
TABLE_NAME = "deployment-logs"

def lambda_handler(event, context):

    message = f"""
    Docker Image pushed successfully to ECR!
    Repository: {event['detail']['repository-name']}
    Image Tag: {event['detail']['image-tag']}
    Time: {datetime.now()}
    """

    # Send email notification via SNS
    sns.publish(
        TopicArn=TOPIC_ARN,
        Subject="Docker Deployment Success",
        Message=message
    )

    # Save log entry to DynamoDB
    table = dynamodb.Table(TABLE_NAME)
    table.put_item(
        Item={
            'id': str(uuid.uuid4()),
            'message': message,
            'timestamp': str(datetime.now())
        }
    )
```

```
)

return {
    'statusCode': 200,
    'body': json.dumps('Notification Sent!')
}
```

Make sure to replace `PASTE_YOUR_SNS_TOPIC_ARN_HERE` with your actual SNS topic ARN before deploying.

Step 15 — Create an EventBridge Rule to Auto-Trigger Lambda

EventBridge watches for ECR image push events and automatically triggers your Lambda function.

Go to **AWS Console** → **Amazon EventBridge** → **Rules** → **Create Rule**.

Step 1 — Basic settings:

Field	Value
Name	ecr-push-trigger
Event Bus	default

Step 2 — Event pattern:

Choose AWS events, select ECR as the event source, and pick ECR Image Action. Use a custom event pattern for best results.

Step 3 — Target:

Set the target as your Lambda function `ecr-image-trigger`.

Now whenever an image is pushed to your ECR repository, EventBridge will fire and invoke the Lambda function automatically.

How the Final Flow Works

1. You push code to GitHub (or trigger a build manually in Jenkins).
2. Jenkins pulls the code, builds the Docker image, and pushes it to ECR.
3. ECR receives the image and emits an event.
4. EventBridge catches that event and triggers the Lambda function.
5. Lambda sends an email notification to your confirmed SNS email address.
6. Lambda also writes a log record to the DynamoDB table.

You can view all deployment logs in the DynamoDB `deployment-logs` table and confirm email alerts are arriving in your inbox.

Application Access

- The Flask app runs on the **DevOps Server** at http://YOUR_DEVOPS_EC2_IP:5000
- Jenkins UI is available on the **Jenkins Server** at http://YOUR_JENKINS_EC2_IP:8080